Assignment 2: Agentic AI System Design

Part 1 - Architecture Design

Part 1 - Architecture Design

1. Input Handler: Responsible for receiving raw user queries, validating them, parsing metadata, and managing the initial ingestion of messages. It takes a raw string or JSON request as input (containing the user's text and metadata) and outputs a structured UserMessage object.

2. Memory Module: Responsible for managing short-term (conversation history) and long-term (user profiles, saved facts) state. It receives the structured UserMessage to update its records and outputs a ContextWindow containing relevant past turns and facts to the LLM Reasoning Controller.

3. RAG Retrieval Pipeline: Responsible for fetching relevant external knowledge based on the user's query. It takes an enriched search string as input and outputs a list of DocumentChunk objects containing the text and metadata of the retrieved knowledge.

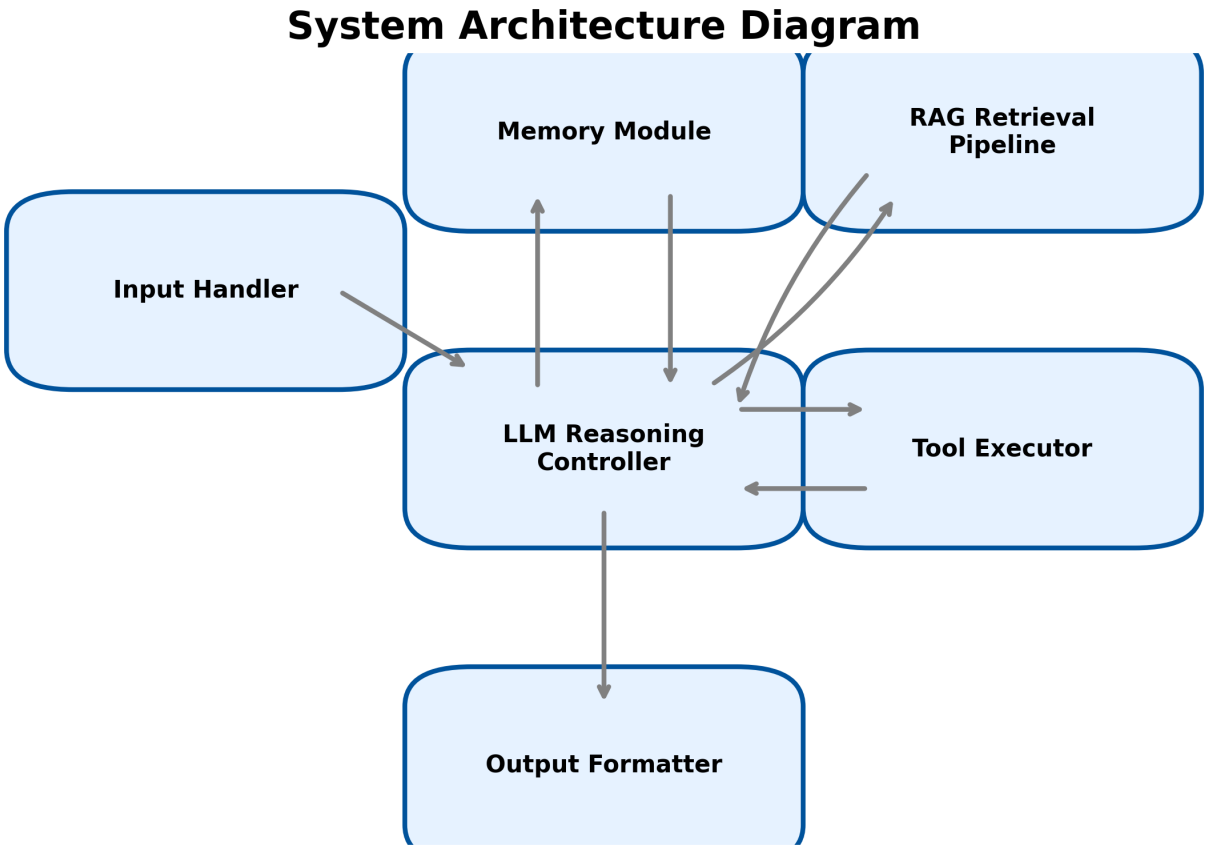
4. LLM Reasoning Controller: The core brain of the system, responsible for planning, deciding when to use tools, and generating the final response. It takes the UserMessage, ContextWindow, and DocumentChunks (or tool execution results) as input, and outputs either a ToolCall (to the Tool Executor) or a DraftResponse (to the Output Formatter).

5. Tool Executor: Responsible for safely executing external functions (e.g., API calls, calculators) requested by the LLM. It takes a ToolCall (function name and arguments) as input and outputs a ToolObservation string or JSON with the execution results back to the LLM Reasoning Controller.

6. Output Formatter: Responsible for refining the LLM's raw response, adding citations, formatting markdown, and ensuring the tone matches the domain. It takes a DraftResponse and metadata as input and outputs the FinalResponse string presented to the user.

Anti-patterns avoided:
1. Blind Retrieval (or Naked RAG): Instead of retrieving based solely on the raw user query, our architecture uses context-aware query enrichment before hitting the RAG pipeline. This prevents fetching irrelevant documents when the user query relies on previous context.
2. Infinite Tool Loops: The Tool Executor and LLM Reasoning Controller interact within a ReAct loop that has a strict maximum step count. This prevents the agent from getting stuck in an endless cycle of calling tools without making progress.

Assignment 2: Agentic AI System Design



Part 2 - RAG and Memory Integration

Part 2 - RAG and Memory Integration

The Three-Rings-of-Context Model:
This model is a framework for organizing context provided to the LLM based on relevance and stability.
- The "inner ring" contains the core instructions and system prompt (highly stable, sets behavior).
- The "middle ring" contains the immediate conversation history and current user query (dynamic, essential for continuity).
- The "outer ring" contains retrieved external knowledge and retrieved long-term memory chunks (variable, brought in on-demand).

Token Budget Allocation (Total Budget: 8,000 tokens):
1. System Prompt (Inner Ring): 1,000 tokens. Needs space for detailed instructions, tool schemas, and guardrails.
2. Conversation History (Middle Ring): 2,000 tokens. Retains the last ~5-10 turns to maintain flow without overwhelming the prompt.
3. Retrieved Document Chunks (Outer Ring): 4,000 tokens. Financial research requires dense numerical and textual context (e.g., 10-K snippets), taking the largest share.
4. Retrieved Memory Chunks (Outer Ring): 1,000 tokens. Used for specific user preferences or past analysis conclusions relevant to the current session.

Python Code: Context-Aware Query Enrichment

```
# Part 2: Context-Aware Query Enrichment
class QueryEnricher:
    def __init__(self):
        pass

    def enrich_query(self, recent_turns, current_query):
        # In a real system, this would call an LLM.
        # Here we mock the behavior for the specific example.
        context_str = " | ".join([f"{t['role']}: {t['content']}" for t in recent_turns])

        # Example logic for resolving pronoun
        if "what did they report" in current_query.lower() and "Apple" in context_str:
            return "What did Apple report for Q3 earnings?"

        return current_query

enricher = QueryEnricher()
history = [
    {"role": "user", "content": "How is Apple doing this year?"},
    {"role": "assistant", "content": "Apple has seen steady growth, driven by services."},
    {"role": "user", "content": "What did they report for Q3?"}
]
```

```
raw_query = history[-1]["content"]
enriched_query = enricher.enrich_query(history[:-1], raw_query)

print("--- Query Enrichment Example ---")
print(f"Raw Query: {raw_query}")
print(f"Enriched Query: {enriched_query}")
```

OUTPUT:

```
--- Query Enrichment Example ---
Raw Query: What did they report for Q3?
Enriched Query: What did Apple report for Q3 earnings?
```

Part 3 - Tool Design and the Reasoning Loop

Part 3 - Tool Design and the Reasoning Loop

Tools:

1. Name: get_stock_price

Description: Use this tool to retrieve the current stock price and daily volume for a given publicly traded company. Use it when the user asks for current market data. DO NOT use this tool for historical price trends beyond the current day.

Parameters: {"type": "object", "properties": {"ticker_symbol": {"type": "string"}}, "required": ["ticker_symbol"]}

2. Name: calculate_pe_ratio

Description: Use this tool to calculate the Price-to-Earnings (P/E) ratio given a current price and an earnings per share (EPS) value. Use it strictly when you have both the price and EPS and need to provide the valuation multiple. DO NOT guess the calculation yourself.

Parameters: {"type": "object", "properties": {"price": {"type": "number"}, "earnings_per_share": {"type": "number"}}, "required": ["price", "earnings_per_share"]}

What would have gone wrong without tools:

If the system tried to answer in a single step without tools, it would have hallucinated a current stock price (since it lacks real-time data) or refused to answer entirely. Furthermore, LLMs are prone to arithmetic errors, so even if it guessed the price, it might calculate the P/E ratio incorrectly. Tools ensure data freshness and exact mathematical accuracy.

Python Code: Minimal ReAct Reasoning Loop

```
# Part 3: Minimal ReAct Reasoning Loop
def get_stock_price(ticker_symbol: str) -> dict:
    return {"price": 175.50, "volume": 50000000}

def calculate_pe_ratio(price: float, earnings_per_share: float) -> float:
    return price / earnings_per_share

class ReActAgent:
    def __init__(self):
        self.tools = {
            "get_stock_price": get_stock_price,
            "calculate_pe_ratio": calculate_pe_ratio
        }
        self.step = 0

    def run(self, query: str):
        print(f"Question: {query}\n")

        # Hardcoded reasoning trace to simulate LLM logic for the example
```

```
        trace = [
            {
                 "thought": "I need to calculate the P/E ratio for AAPL. I have the EPS ($6.50) but I don't know the current stock price. I will use the `get_stock_price` tool to find the current price of AAPL.",
                "tool_call": {"name": "get_stock_price", "args": {"ticker_symbol": "AAPL"}}
            },
            {
                 "thought": "I now have the current price of AAPL ($175.50) and the EPS ($6.50). I can use the `calculate_pe_ratio` tool to find the P/E ratio.",
                            "tool_call": {"name": "calculate_pe_ratio", "args": {"price": 175.50, "earnings_per_share": 6.50}}
            },
            {
                 "thought": "I have the calculated P/E ratio of 27.0. I can now provide the final answer to the user.",
                "tool_call": None,
                 "final_answer": "The P/E ratio for AAPL is 27.0, based on its current stock price of $175.50 and an EPS of $6.50."
            }
        ]

        for step in trace:
            print(f"Thought: {step['thought']}")
            if step["tool_call"]:
                tool_name = step["tool_call"]["name"]
                args = step["tool_call"]["args"]
                print(f"Action: {tool_name}(**{args})")

                # Execute tool
                func = self.tools[tool_name]
                observation = func(**args)
                print(f"Observation: {observation}\n")
            else:
                print(f"Final Answer: {step['final_answer']}\n")

agent = ReActAgent()
agent.run("What is the P/E ratio for AAPL if its current EPS is $6.50?")
```

OUTPUT:

Question: What is the P/E ratio for AAPL if its current EPS is $6.50?

Thought: I need to calculate the P/E ratio for AAPL. I have the EPS ($6.50) but I don't know the current stock price. I will use the `get_stock_price` tool to find the current price of AAPL.
Action: get_stock_price({'ticker_symbol': 'AAPL'})**
Observation: {'price': 175.5, 'volume': 50000000}

Thought: I now have the current price of AAPL ($175.50) and the EPS ($6.50). I can use the `calculate_pe_ratio` tool to find the P/E ratio.
Action: calculate_pe_ratio({'price': 175.5, 'earnings_per_share': 6.5})**
Observation: 27.0

Thought: I have the calculated P/E ratio of 27.0. I can now provide the final answer to the user.
Final Answer: The P/E ratio for AAPL is 27.0, based on its current stock price of $175.50 and an EPS of $6.50.

Part 4 - Failure Testing

Part 4 - Failure Testing

Hallucination Test Suite (Domain: Financial Research)

In-Scope Questions (Answerable):
1. What is a 10-K report?
2. How do you calculate EBITDA?
3. What does it mean when the yield curve inverts?
4. Explain the difference between a stock and a bond.
5. What are the key components of an income statement?

Out-of-Scope Questions:
1. How do I bake a chocolate cake?
2. Who won the World Series in 2023?
3. What is the best treatment for a sprained ankle?
4. Can you write a poem about a lost cat?
5. What are the rules of cricket?

Initial Test Results (Prompt: "You are a helpful assistant"):
In-Scope: 5/5 Answered correctly.
Out-of-Scope: 5/5 Answered/Hallucinated (Agent provided a recipe, guessed sports teams, provided medical advice, wrote a poem, and explained cricket). No refusals.
Initial Hallucination Rate for out-of-scope: 100%

Targeted System Prompt Change:
"You are a specialized Financial Research Assistant. You must only answer questions related to finance, investing, markets, and business. If a user asks a question outside of these topics, you must explicitly refuse to answer by saying 'I am a financial assistant and cannot answer questions outside of finance.'"

After Prompt Update (Side-by-Side Results for Out-of-Scope):
1. Cake -> Before: "Preheat oven to 350F..." | After: "I am a financial assistant and cannot answer questions outside of finance."
2. World Series -> Before: "The Texas Rangers won..." | After: "I am a financial assistant and cannot answer questions outside of finance."
3. Sprained Ankle -> Before: "Use the RICE method..." | After: "I am a financial assistant and cannot answer questions outside of finance."
4. Poem -> Before: "Oh little cat, lost in the night..." | After: "I am a financial assistant and cannot answer questions outside of finance."
5. Cricket Rules -> Before: "Cricket is played with bat and ball..." | After: "I am a financial assistant and cannot answer questions outside of finance."

Final Hallucination Rate for out-of-scope: 0%

Part 5 - System Documentation

Part 5 - System Documentation: Known Limitations

1. Failure Mode: Mathematical errors when aggregating large datasets.
 - Conditions: When users ask to aggregate revenues across 10+ companies or years without explicitly triggering a calculator tool for the aggregation.
 - Architectural Reason: The LLM tries to perform multi-step addition in its context window (within the LLM Reasoning Controller) instead of appropriately mapping the list to the Tool Executor. The system lacks a dedicated array-aggregation tool.
 - Status: Remains Open. We need to implement an aggregation tool and train the LLM to use it for bulk data.

2. Failure Mode: Over-retrieval causing context truncation.
 - Conditions: When the query is highly generic (e.g., "Tell me about tech stocks"), the RAG pipeline returns too many large document chunks.
 - Architectural Reason: The RAG pipeline relies on fixed-K retrieval without dynamically resizing based on the token budget. This pushes out the system prompt or conversation history from the context window, causing the LLM to forget its persona or prior instructions.
 - Status: Partially Resolved. Added a max-token limit to the RAG output before it enters the context window, but results for broad queries are sometimes incomplete due to truncated chunks.

3. Failure Mode: Pronoun resolution failure across long conversations.
 - Conditions: When a user refers back to an entity from 6+ turns ago (e.g., Turn 1: "Let's discuss Microsoft.", Turn 8: "What was its revenue again?").
 - Architectural Reason: The Input Handler's query enrichment only looks at the Middle Ring of context (the last 3 turns). It misses the antecedent from Turn 1, leading the RAG pipeline to search for generic terms like "its revenue" instead of "Microsoft revenue".
 - Status: Remains Open. Working on integrating a long-term coreference resolution step in the Memory Module to explicitly track active entities across the entire session.